

Universidad Nacional Autónoma de México.



Facultad de Ingeniería.

Sistemas Distribuidos.

Profesor: Ing. Óscar Manuel Ruíz Hurtado.

Semestre 2027-1

Laboratorio S9.

Mexi Banco: que el aviso no tumbe la transferencia.

Grupo: 2

Alumna:

Valencia Lerdo Dalia Jimena

Cd. Universitaria a 24 de octubre de 2026

1. **El problema.**

El cliente recibe un error 500 porque, después de que el cargo, el abono y los asientos ya se hicieron, la saga llama por HTTP a notificación y se queda esperando su respuesta. Como notificación está apagada, esa llamada falla antes de que la transferencia se marque COMPLETADA. Por eso el dinero sí se movió, pero la transferencia quedó en ABONADA.

Imagen 1. Exchanges de RabbitMQ antes de empezar.

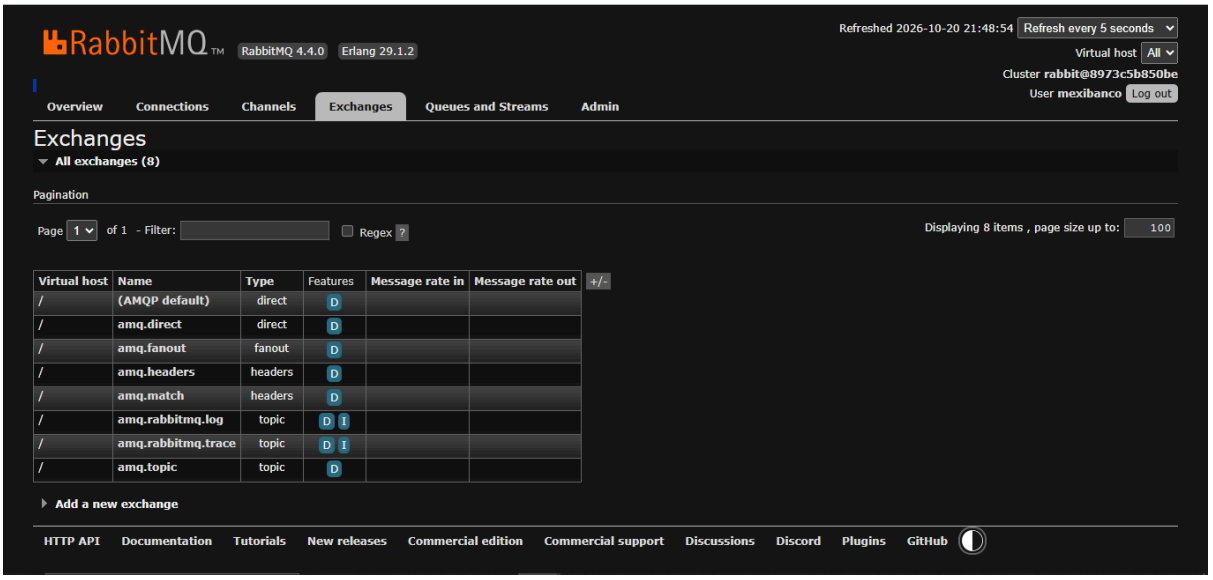


Imagen 2. Colas de RabbitMQ antes de empezar.

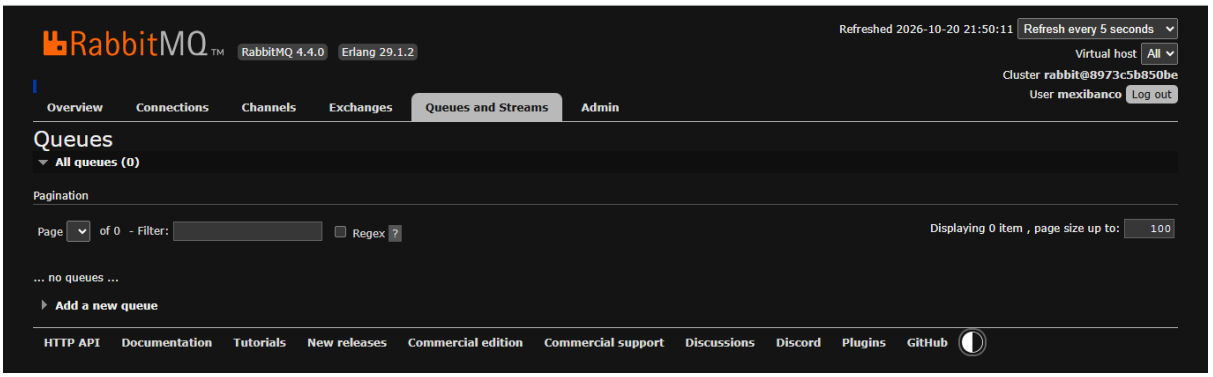


Imagen 3. Notificación apagada y la transferencia de 50 responde 500.

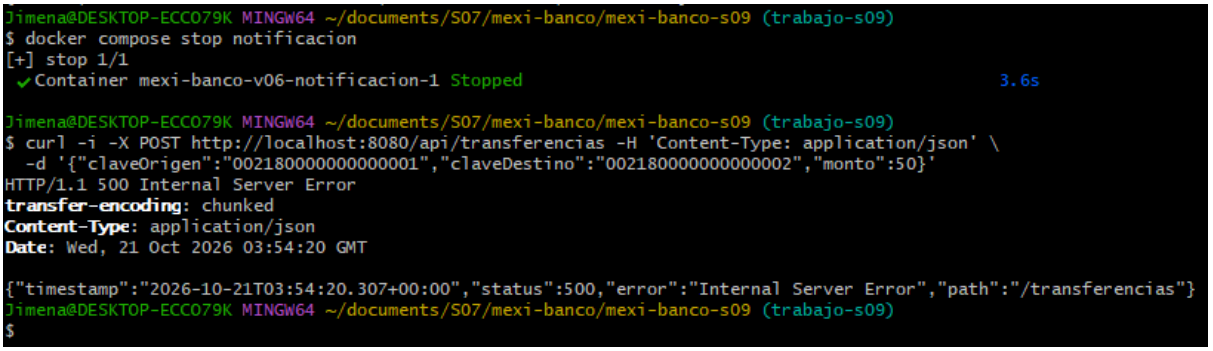
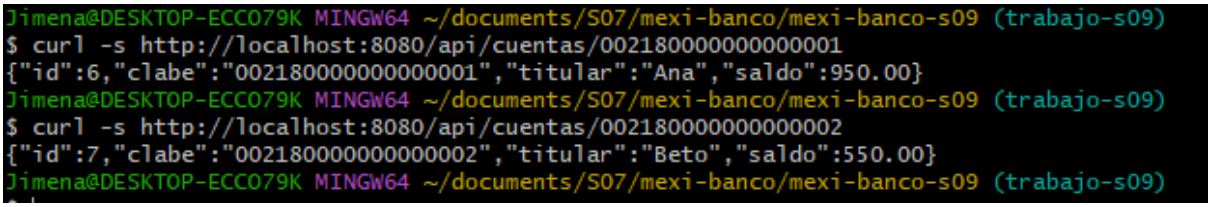


Imagen 4. La transferencia quedó atorada en ABONADA.



Imagen 5. Saldos después del error. (Ana 950 y Beto 550).



2. **Publicar.**

Para que el aviso no tumbe la transferencia cambié la saga, ya no llama a notificación por HTTP, ahora deja un evento en RabbitMQ. Todo el código de esta sección está en el servicio transferencia.

eventoId. Cada transferencia nace con un UUID que se guarda junto con su fila. Si el evento se publica dos veces lleva el mismo id, y eso nos sirve después para detectar el duplicado. Las filas viejas, que no lo traen, reciben uno al leerse (@PostLoad).

EventoTransferencia. Es el mensaje y funciona a forma de contrato. Lleva el id, el tipo (TransferenciaRealizada o TransferenciaRevertida), las CLABEs, el monto y los saldos. Usé un record propio y no la entidad para que un cambio en la base no rompa a quien escucha. Cada tipo tiene su routing key: transferencia.realizada y transferencia.revertida.

Exchange transferencias. Lo declaré de tipo topic y durable. Transferencia solo publica ahí y no sabe quién escucha; las colas son de quien consume. También configuré que los mensajes viajen como JSON.

application.yml. Activé los publisher confirms y puse un connection-timeout de 2 segundos para que no se quede colgada 60 segundos si RabbitMQ no contesta. Con mandatory y publisher-returns, un mensaje que ninguna cola recibe regresa en lugar de tirarse en silencio.

PublicadorDeEventos. Manda el evento con modo persistente y con el eventoId como messageId. Después espera hasta 5 segundos la confirmación del broker y lanza EventoNoPublicadoException si hay timeout, si responde nack o si ninguna cola recibió el mensaje.

TransferenciaService. Primero la base dice COMPLETADA (o COMPENSADA) y hasta después se publica el evento. Si publicar falla, solo se registra en el log, así el cliente no recibe un error por un aviso. Las dos llamadas HTTP a notificación se quitaron.

¿Qué garantiza el publisher confirm y qué no? Garantiza que RabbitMQ recibió y guardó el mensaje. No garantiza que notificación lo haya leído, ni que no se pierda si transferencia muere justo entre guardar la transferencia y publicar el evento.

Con notificación apagada y todavía sin cola, la transferencia respondió 201, pero el log dijo que el evento no se publicó porque ninguna cola lo recibió (NO_ROUTE). Un exchange no guarda mensajes, así que ese aviso se perdió. Se arregló en la siguiente sección, cuando notificación declaró su propia cola.

Imagen 6. Transferencia de 50 con 201 y el log con NO_ROUTE



3. Desacople en tiempo.

Con notificación apagada hice tres transferencias (de 11, 12 y 13) y las tres respondieron 201. Los tres eventos quedaron esperando en la cola durable notificacion.transferencias, que mostraba 3 Ready y 0 consumidores. Al encender notificación la cola bajó a 0 y las bandejas de Ana y de Beto recibieron sus avisos. Así el productor y el consumidor ya no tienen que estar vivos al mismo tiempo.

Imagen 7. Tres transferencias con notificación apagada: tres 201 y 3 mensajes en la cola.

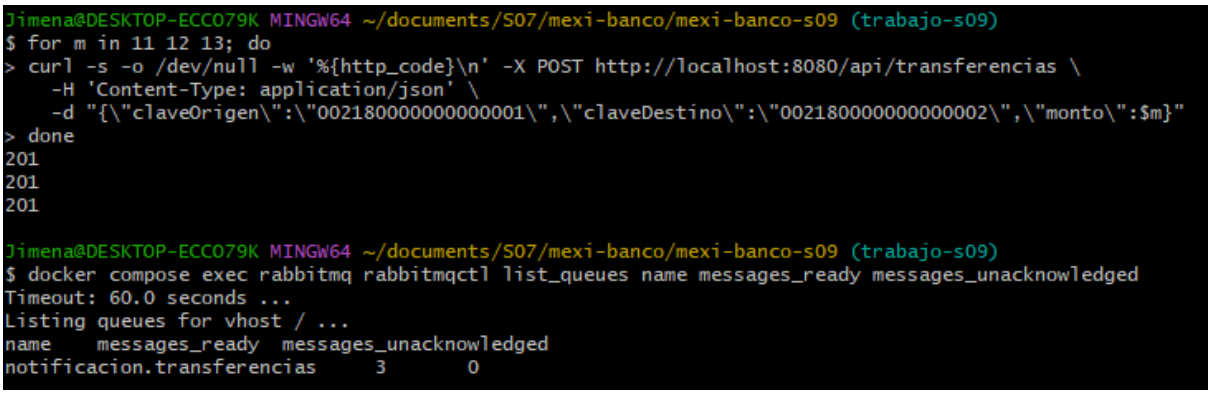


Imagen 8. Bandeja de Beto al volver notificación: avisos de 11, 12 y 13.



Para probar la revertida provoqué una falla a propósito en el abono hacia Beto (FALLAR_ABONO_A_CLABE). La saga devolvió el cargo a Ana, pero el asiento de la devolución falló porque la tabla movimiento tenía una restricción vieja (movimiento_tipo_check) que no aceptaba el tipo DEVOLUCION_TRANSFERENCIA. La transferencia quedó en COMPENSACION_PENDIENTE y el reintento programado de la S08 la siguió intentando. Después de corregir la restricción, la transferencia 16 pasó a

COMPENSADA y a Ana le llegó el aviso de devolución por 150, con su saldo de vuelta en 864. Hasta la S08 nadie le avisaba al cliente que su transferencia se había revertido.

Imagen 9. Abono fallando a propósito: 502 y transferencia en COMPENSACION_PENDIENTE.

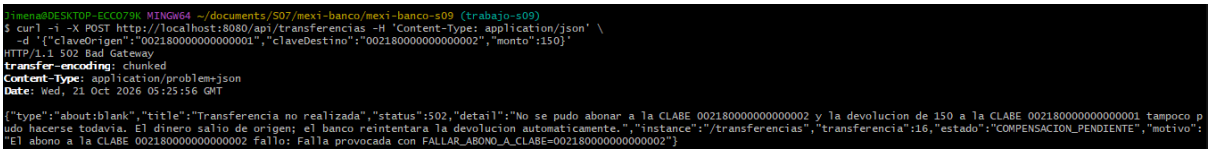
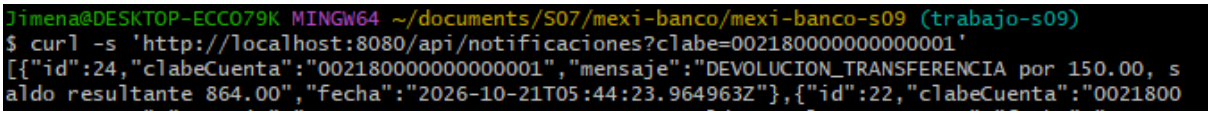


Imagen 10. Bandeja de Ana con el aviso de devolución por 150 (saldo 864)



4. El duplicado.

RabbitMQ entrega los mensajes al menos una vez, así que un evento puede llegar dos veces. Lo comprobé con notificación apagada: hice una transferencia, leí su mensaje desde la API de RabbitMQ sin consumirlo y publiqué otra vez el mismo evento, con el mismo eventoId. La cola quedó con 2 Ready. Al encender notificación procesó los dos, el log muestra la transferencia 22 dos veces con el mismo eventoId y cada entrega guardó 2 avisos. La bandeja de Beto quedó con dos avisos iguales de 45, con el mismo saldo resultante (821).

Antes del arreglo

Imagen 11. Cola con 2 mensajes: el original y el duplicado.

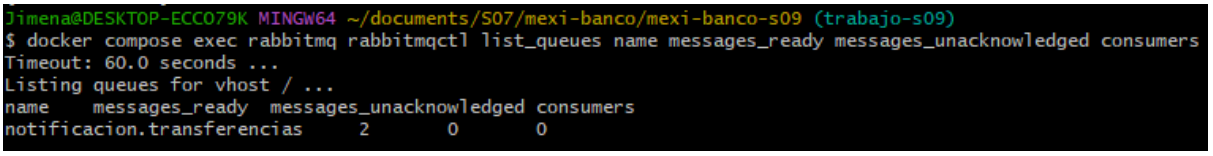
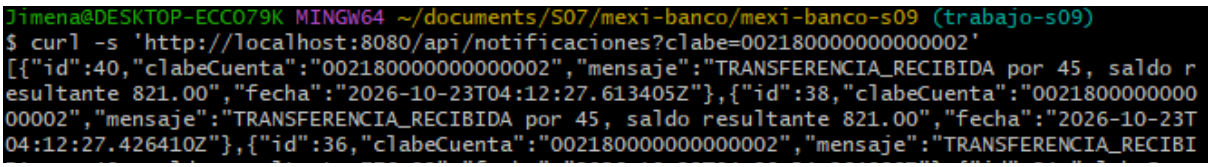


Imagen 12. Log con la transferencia 22 procesada dos veces (mismo eventoId).



Imagen 13. Bandeja de Beto con el aviso de 45 repetido (mismo saldo 821).

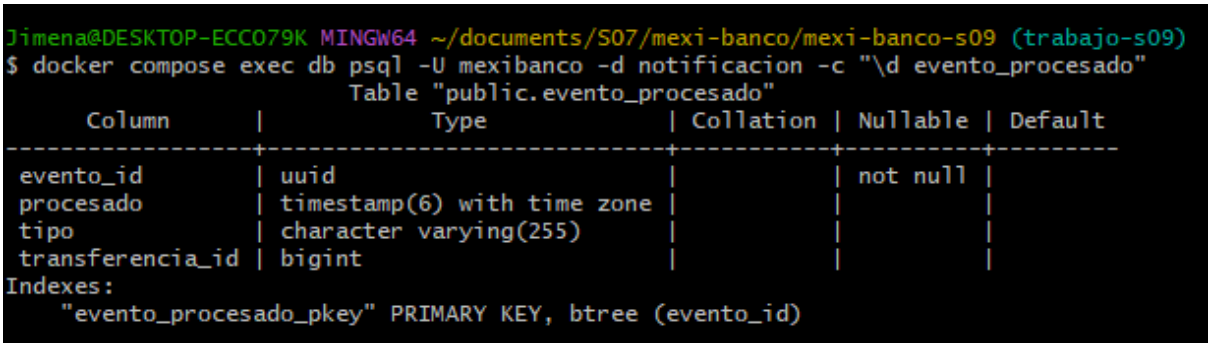


El arreglo.

Creé la tabla evento_procesado, con el eventoId como llave primaria, así que la unicidad la impone Postgres y no mi código. Antes de guardar los avisos hago un INSERT … ON CONFLICT DO NOTHING: regresa 1 si el id era nuevo y 0 si ya existía. Lo hice en una sola sentencia porque con "consultar y luego insertar" una segunda entrega podría colarse entre las dos llamadas. Tampoco atrapo la excepción de llave duplicada, porque en Postgres esa violación deja la transacción abortada. Si el id ya estaba, el método regresa una lista vacía y el consumidor confirma el mensaje de todos modos.

Todo va en la misma transacción que los avisos. Si algo falla después de marcar el evento, el rollback deshace la marca y los avisos juntos, y el siguiente intento empieza de cero. Si la marca fuera aparte, un fallo dejaría el evento marcado sin avisos guardados y ese aviso se perdería.

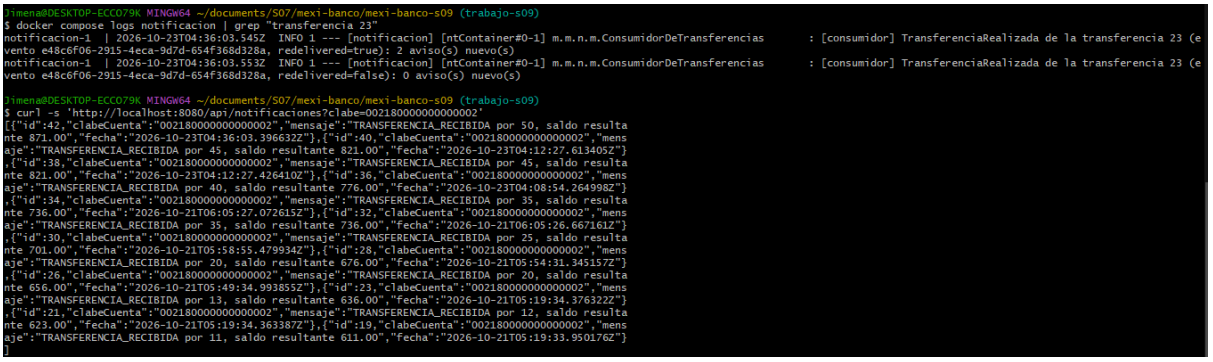
Imagen 14. Tabla evento_procesado con evento_id como llave primaria**.**



Después del arreglo.

Repetí el experimento con la transferencia 23 (monto 50). El log muestra dos entregas del mismo eventoId: la primera guardó 2 avisos y la segunda dijo 0 aviso(s) nuevo(s). En la bandeja de Beto el aviso de 50 aparece una sola vez (id 42).

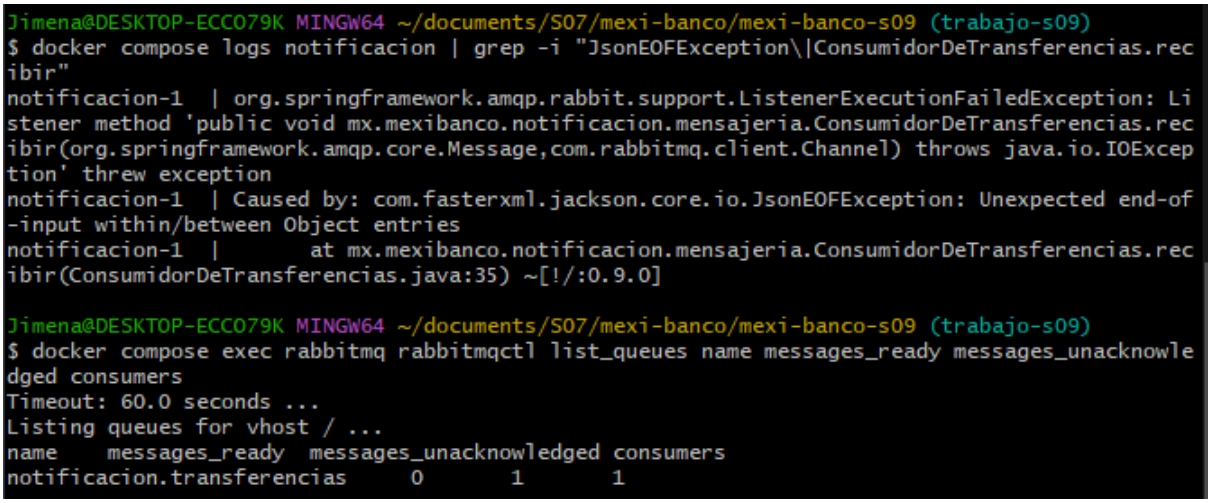
Imagen 15. Después del arreglo: segunda entrega con 0 avisos nuevos y un solo aviso de 50.



5. **La DLQ.**

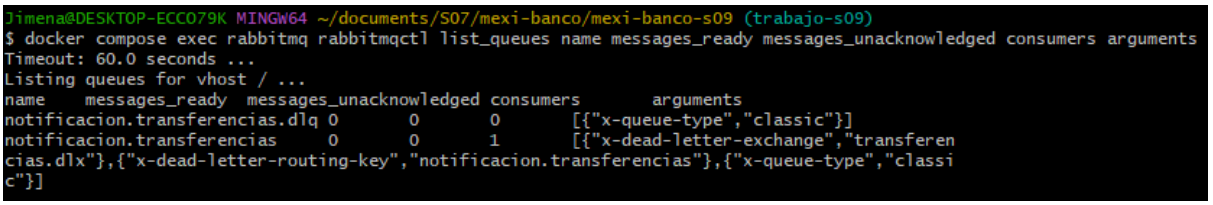
Publiqué en el exchange transferencias un mensaje con JSON incompleto. Sin DLQ, el consumidor lanzó una JsonEOFException al leerlo, antes de confirmarlo. El mensaje no rebotó ni tiró el servicio: se quedó trabado en Unacked (la cola marcaba 0 Ready, 1 Unacked y 1 consumidor), y las transferencias normales siguieron generando avisos.

Imagen 16. Antes de la DLQ: excepción al leer el JSON y 1 mensaje trabado en Unacked.



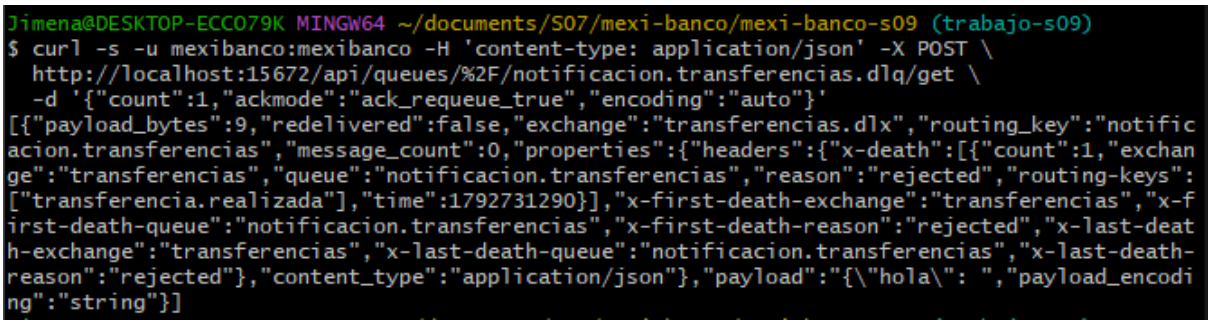
Para arreglarlo declaré el exchange transferencias.dlx y la cola notificacion.transferencias.dlq, y le agregué a la cola principal los argumentos x-dead-letter-exchange y x-dead-letter-routing-key. En el consumidor, si el JSON no se puede leer, hago basicNack con requeue en false. Así es el broker, y no mi servicio, quien mueve el mensaje a la DLQ. Solo mandé a la DLQ el mensaje ilegible, que nunca se va a poder procesar; un fallo pasajero, como la base caída, conviene reintentarlo. Como la cola ya existía sin esos argumentos, RabbitMQ marcó PRECONDITION_FAILED, y tuve que borrarla (rabbitmqctl delete_queue) para que notificación la declarara de nuevo.

Imagen 17. Colas con sus argumentos: la principal apunta al DLX y existe la DLQ.



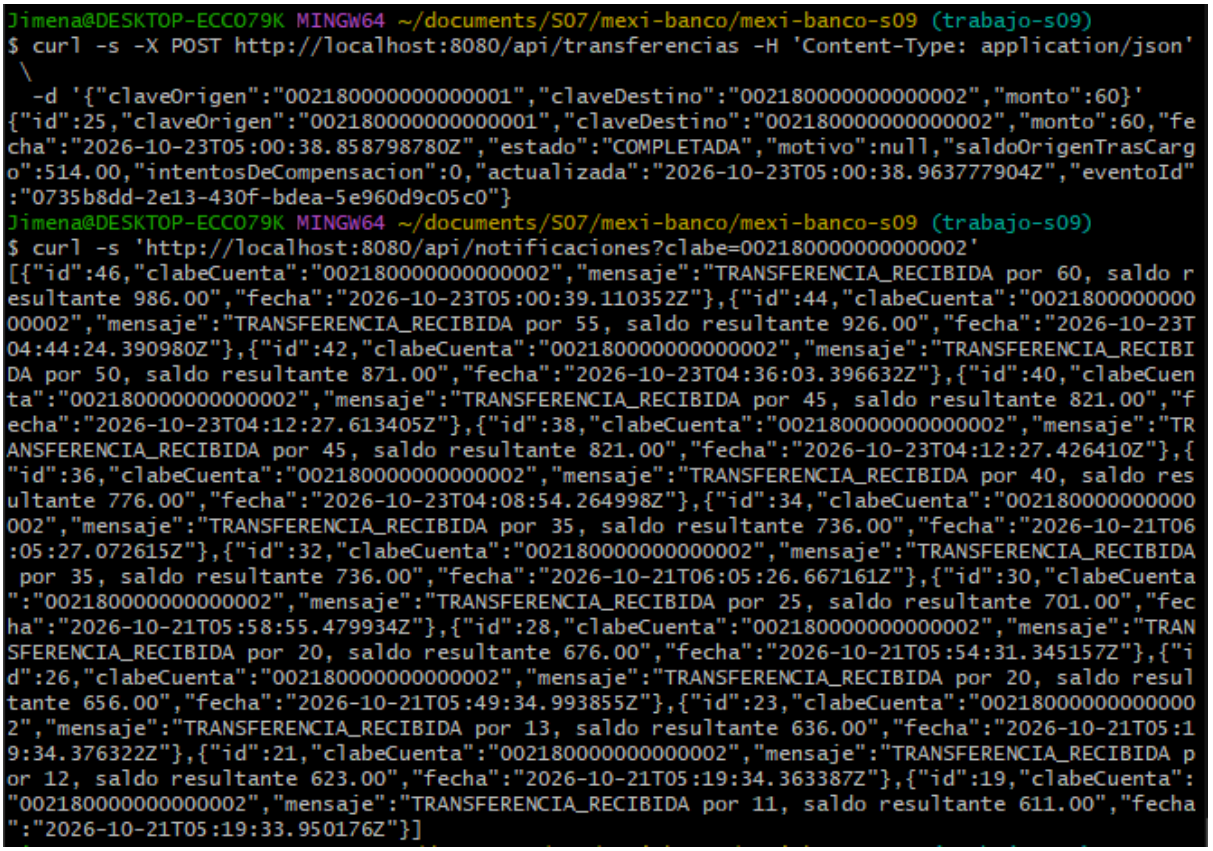
Repetí la prueba con el mismo mensaje malo. La cola principal quedó sin mensajes trabados y el mensaje apareció en la DLQ. El x-death indica que fue rechazado (reason: rejected) desde notificacion.transferencias. La transferencia siguiente (monto 60) sí generó su aviso.

Imagen 18. Mensaje en la DLQ con x-death (reason: rejected).



```
Jimena@DESKTOP-ECCO79K MINGW64 ~/documents/S07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s -u mexibanco:mexibanco -H 'content-type: application/json' -X POST \
  http://localhost:15672/api/queues/%2F/notificacion.transferencias.dlq/get \
  -d '{"count":1,"ackmode":"ack_requeue_true","encoding":"auto"}'
[{"payload_bytes":9,"redelivered":false,"exchange":"transferencias.dlx","routing_key":"notific
acion.transferencias","message_count":0,"properties":{"headers":{"x-death":[{"count":1,"exchan
ge":"transferencias","queue":"notificacion.transferencias","reason":"rejected","routing-keys":
["transferencia.realizada"],"time":1792731290}],"x-first-death-exchange":"transferencias","x-f
irst-death-queue":"notificacion.transferencias","x-first-death-reason":"rejected","x-last-deat
h-exchange":"transferencias","x-last-death-queue":"notificacion.transferencias","x-last-death-
reason":"rejected"},"content_type":"application/json"},"payload":"{\"hola\": ","payload_encodi
ng":"string"}]
```

Imagen 19. Transferencia de 60 normal después del mensaje malo, con su aviso.



```
Jimena@DESKTOP-ECCO79K MINGW64 ~/documents/S07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json'
 \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":60}'
{"id":25,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":60,"fe
cha":"2026-10-23T05:00:38.858798780Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCarg
o":514.00,"intentosDeCompensacion":0,"actualizada":"2026-10-23T05:00:38.963777904Z","eventoId"
:"0735b8dd-2e13-430f-bdea-5e960d9c05c0"}
Jimena@DESKTOP-ECCO79K MINGW64 ~/documents/S07/mexi-banco/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":46,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 60, saldo r
esultante 986.00","fecha":"2026-10-23T05:00:39.110352Z"},{"id":44,"clabeCuenta":"0021800000000
00002","mensaje":"TRANSFERENCIA_RECIBIDA por 55, saldo resultante 926.00","fecha":"2026-10-23T
04:44:24.390980Z"},{"id":42,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBI
DA por 50, saldo resultante 871.00","fecha":"2026-10-23T04:36:03.396632Z"},{"id":40,"clabeCuen
ta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 45, saldo resultante 821.00","f
echa":"2026-10-23T04:12:27.613405Z"},{"id":38,"clabeCuenta":"002180000000000002","mensaje":"TR
ANSFERENCIA_RECIBIDA por 45, saldo resultante 821.00","fecha":"2026-10-23T04:12:27.426410Z"},{
"id":36,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 40, saldo res
ultante 776.00","fecha":"2026-10-23T04:08:54.264998Z"},{"id":34,"clabeCuenta":"002180000000000
002","mensaje":"TRANSFERENCIA_RECIBIDA por 35, saldo resultante 736.00","fecha":"2026-10-21T06
:05:27.072615Z"},{"id":32,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA
 por 35, saldo resultante 736.00","fecha":"2026-10-21T06:05:26.667161Z"},{"id":30,"clabeCuenta
":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 25, saldo resultante 701.00","fec
ha":"2026-10-21T05:58:55.479934Z"},{"id":28,"clabeCuenta":"002180000000000002","mensaje":"TRAN
SFERENCIA_RECIBIDA por 20, saldo resultante 676.00","fecha":"2026-10-21T05:54:31.345157Z"},{"i
d":26,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 20, saldo resul
tante 656.00","fecha":"2026-10-21T05:49:34.993855Z"},{"id":23,"clabeCuenta":"00218000000000000
2","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-21T05:1
9:34.376322Z"},{"id":21,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA p
or 12, saldo resultante 623.00","fecha":"2026-10-21T05:19:34.363387Z"},{"id":19,"clabeCuenta":
"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","fecha
":"2026-10-21T05:19:33.950176Z"}]
```

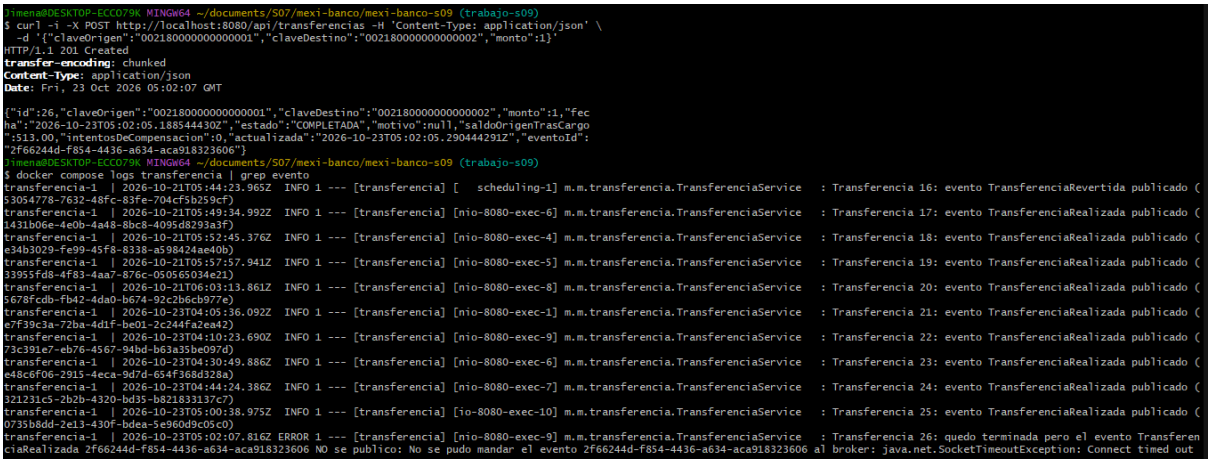
6. Preguntas de cierre.

1. Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso? ¿Por qué no lo arregla el publisher confirm?

La transferencia termina bien: responde 201 y queda COMPLETADA, aunque tarda unos 2 segundos más por el connection-timeout. El evento no se publica y el log lo avisa con un error (Connect timed out). Cuando RabbitMQ vuelve nadie lo manda, así que ese aviso no llega nunca. El publisher confirm no lo arregla porque solo me dice si el broker recibió el mensaje, y con el broker apagado no hay a quién preguntar. El evento tampoco queda guardado en ningún lado para reintentarlo. Es el problema de la doble escritura: la base dice que la transferencia se completó y el broker nunca se enteró, y no existe una

transacción que una a los dos. Se arreglaría con un Outbox: guardar el evento en la misma transacción que el estado final y publicarlo después.

Imagen 20. RabbitMQ apagado: 201 COMPLETADA y el log con el evento NO publicado.



2. ¿Por qué el ack va después de guardar y no antes?

Si el ack va después de guardar y notificación muere en medio, el broker no recibe el ack y vuelve a entregar el mensaje (redelivered=true). El aviso puede llegar dos veces, pero no se pierde. Si el ack fuera antes de guardar y notificación muere en medio, el broker ya borró el mensaje y el aviso se pierde. Preferí que nunca se perdiera, porque un repetido se arregla con idempotencia por eventoId y uno perdido no tiene arreglo.

3. Elección, porque, sacrifico.

Elección: Avisar por mensajería asíncrona en lugar de por HTTP.
Porque: Así el aviso ya no tumba la transferencia y notificación puede estar apagada sin afectar al cliente.
Sacrifico: Que el aviso ya no es inmediato ni está garantizado, puede llegar tarde, puede llegar repetido y puede perderse si RabbitMQ está caído justo cuando se completa la transferencia.